

SQL 기초

MySQL로 배우는 데이터베이스 개론과 실습

실습 자료 링크

- 실습자료 다운로드 링크

MySQL과 MariaDB

- MySQL은 1995년 발표된 오픈소스 DBMS로 스웨덴의 MySQL AB사 제품
- 이후 썬 마이크로시스템사에 인수 그 이후 오라클사에 인수
- 인수 이후 오픈소스 기반이였으나 MySQL에 비호의적
- 주 개발자였던 마이클 몬티 와이드니우스(Michael Monty Widenius)가 회사를 나와서 MySQL 5.5 기반에서 MariaDB라는 오픈소스 버전을 출시

MySQL	
최초 개발사	MySQL AB
개발사	오라클
발표일	1995년 5월 23일
라이선스	GPL v2 (커뮤니티 에디션) 상용 라이선스 (표준, 엔터프라이즈, 클러스터)
링크	

MariaDB	
개발자	MariaDB 재단
발표일	2009년 10월 29일
계열	MySQL
라이선스	GPL v2 LGPL v2.1 (클라이언트 라이브러리)
홈페이지	
소스 코드	

SQL 개요

❖ 주요 키워드

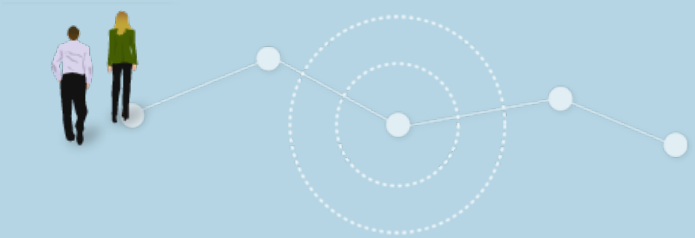
- SELECT
- FROM
- WHERE
- GROUP BY
- HAVING
- ORDER BY
- JOIN
- 기타

❖ 주석 종류

- # 한 줄의 경우
- -- 한 줄의 경우
- /* 여러줄의 주석일 경우 */

데이터 조작어 - 검색

1. 두 개 이상 테이블에서 SQL 질의



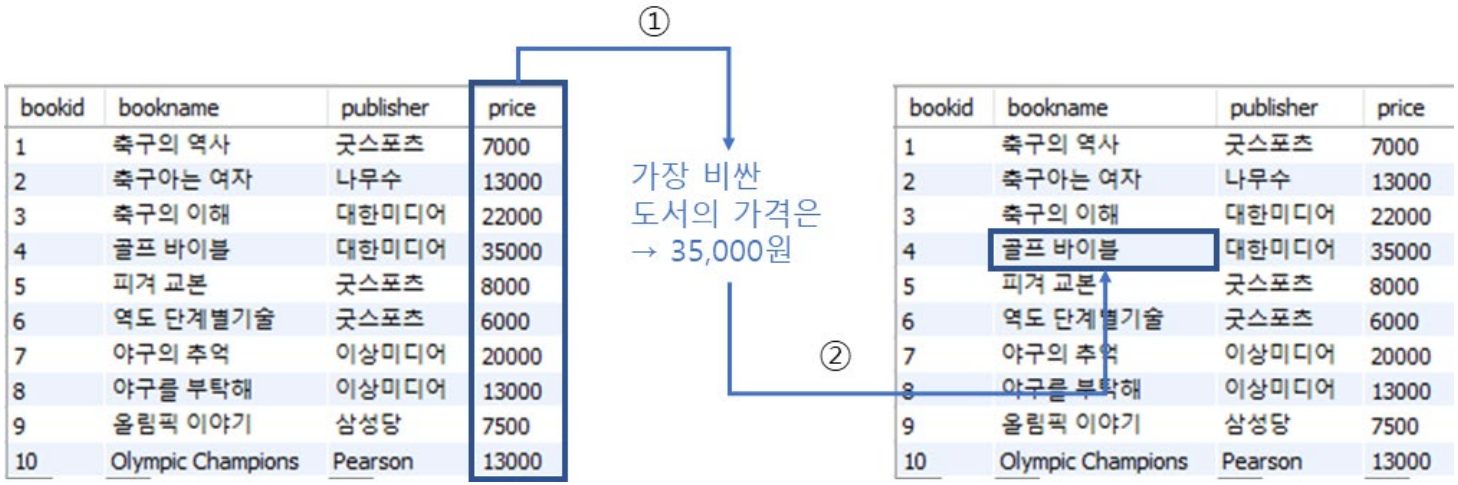
두 개 이상 테이블에서 SQL 질의

❖ 부속질의_SQL 문 내에 또 다른 SQL 문을 작성해보자

질의 3-28 가장 비싼 도서의 이름을 보이시오.

```
SELECT    bookname
FROM      Book
WHERE     price = ( SELECT MAX(price)
                    FROM Book);
```

bookname
골프 바이블



bookid	bookname	publisher	price
1	축구의 역사	굿스포츠	7000
2	축구아는 여자	나무수	13000
3	축구의 이해	대한미디어	22000
4	골프 바이블	대한미디어	35000
5	피겨 교본	굿스포츠	8000
6	역도 단계별기술	굿스포츠	6000
7	야구의 추억	이상미디어	20000
8	야구를 부탁해	이상미디어	13000
9	올림픽 이야기	삼성당	7500
10	Olympic Champions	Pearson	13000

그림 3-21 **부속질의의 실행 순서**

두 개 이상 테이블에서 SQL 질의

❖ 부속질의_SQL 문 내에 또 다른 SQL 문을 작성해보자

질의 3-29 도서를 구매한 적이 있는 고객의 이름을 검색하시오.

```
SELECT    name
FROM      Customer
WHERE     custid IN (SELECT    custid
                     FROM      Orders);
```

name
박지성
김연아
장미란
추신수

질의 3-30 대한미디어에서 출판한 도서를 구매한 고객의 이름을 보이시오.

```
SELECT    name
FROM      Customer
WHERE     custid IN (SELECT    custid
                     FROM      Orders
                     WHERE     bookid IN (SELECT    bookid
                                          FROM      Book
                                          WHERE     publisher='대한미디어'));
```

name
박지성

두 개 이상 테이블에서 SQL 질의

❖ 부속질의_SQL 문 내에 또 다른 SQL 문을 작성해보자

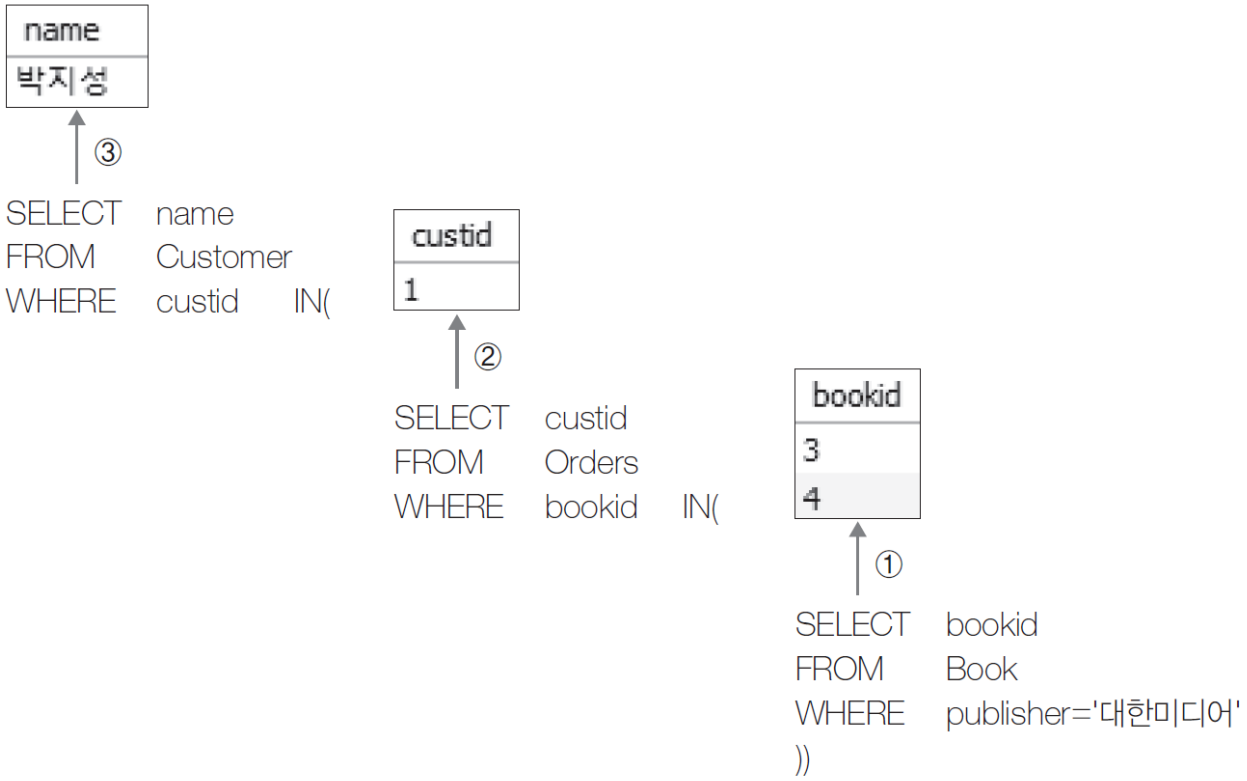
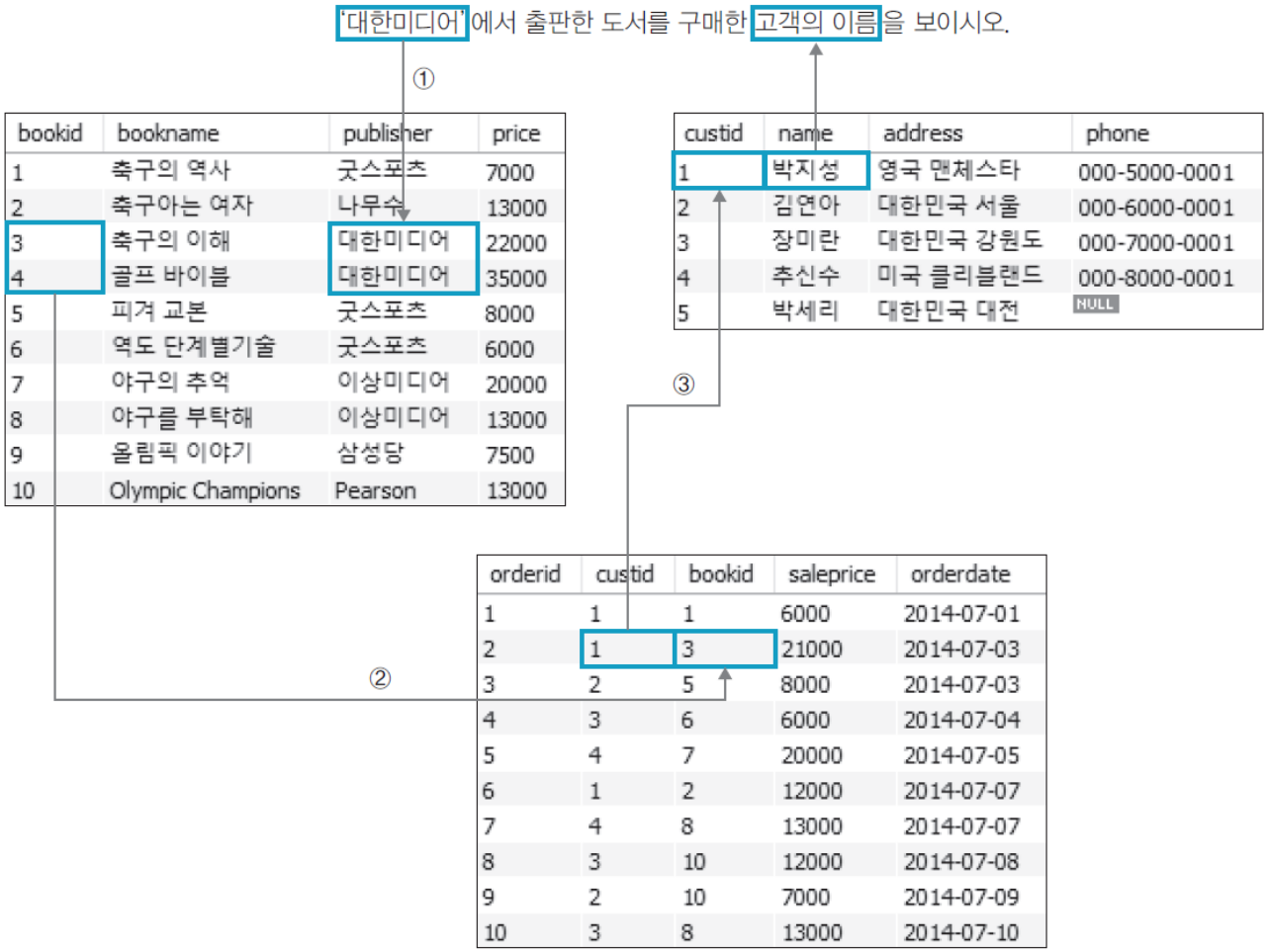


그림 3-22 3단계 부속질의의 실행 순서

두 개 이상 테이블에서 SQL 질의

❖ 부속질의_SQL 문 내에 또 다른 SQL 문을 작성해보자



bookid	bookname	publisher	price
1	축구의 역사	굿스포츠	7000
2	축구아는 여자	나무수	13000
3	축구의 이해	대한미디어	22000
4	골프 바이블	대한미디어	35000
5	피겨 교본	굿스포츠	8000
6	역도 단계별기술	굿스포츠	6000
7	야구의 추억	이상미디어	20000
8	야구를 부탁해	이상미디어	13000
9	올림픽 이야기	삼성당	7500
10	Olympic Champions	Pearson	13000

custid	name	address	phone
1	박지성	영국 맨체스타	000-5000-0001
2	김연아	대한민국 서울	000-6000-0001
3	장미란	대한민국 강원도	000-7000-0001
4	추신수	미국 클리블랜드	000-8000-0001
5	박세리	대한민국 대전	NULL

orderid	custid	bookid	saleprice	orderdate
1	1	1	6000	2014-07-01
2	1	3	21000	2014-07-03
3	2	5	8000	2014-07-03
4	3	6	6000	2014-07-04
5	4	7	20000	2014-07-05
6	1	2	12000	2014-07-07
7	4	8	13000	2014-07-07
8	3	10	12000	2014-07-08
9	2	10	7000	2014-07-09
10	3	8	13000	2014-07-10

그림 3-23 **3단계 부속질의의 실행 순서와 데이터 예**

두 개 이상 테이블에서 SQL 질의

❖ 부속질의_SQL 문 내에 또 다른 SQL 문을 작성해보자

- **상관 부속질의(correlated subquery)는 상위 부속질의의 투플을 이용하여 하위 부속질의를 계산함. 즉 상위 부속질의와 하위 부속질의가 독립적이지 않고 서로 관련을 맺고 있음.**

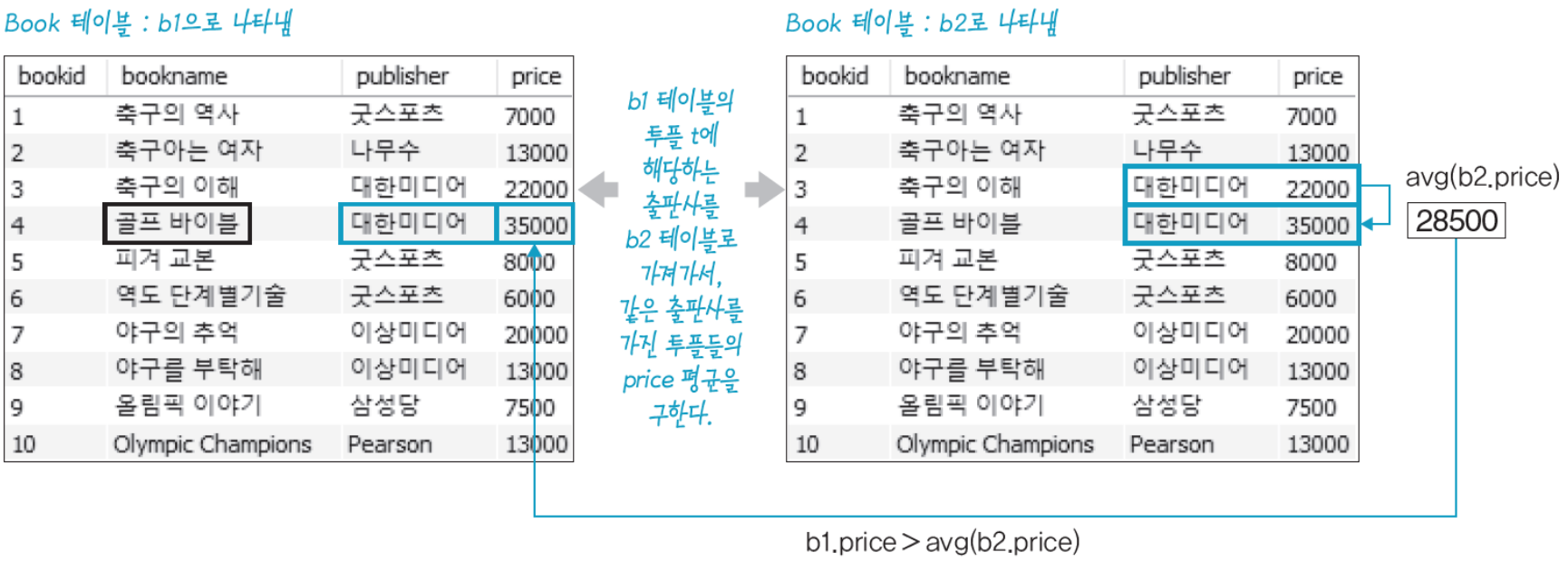
질의 3-31 출판사별로 출판사의 평균 도서 가격보다 비싼 도서를 구하시오.

```
SELECT    b1.bookname
FROM      Book b1
WHERE     b1.price > (SELECT    avg(b2.price)
                      FROM      Book b2
                      WHERE     b2.publisher=b1.publisher);
```

bookname
골프 바이블
피겨 교본
야구의 추억

두 개 이상 테이블에서 SQL 질의

❖ 부속질의_SQL 문 내에 또 다른 SQL 문을 작성해보자



bookid	bookname	publisher	price
1	축구의 역사	굿스포츠	7000
2	축구아는 여자	나무수	13000
3	축구의 이해	대한미디어	22000
4	골프 바이블	대한미디어	35000
5	피겨 교본	굿스포츠	8000
6	역도 단계별기술	굿스포츠	6000
7	야구의 추억	이상미디어	20000
8	야구를 부탁해	이상미디어	13000
9	올림픽 이야기	삼성당	7500
10	Olympic Champions	Pearson	13000

bookid	bookname	publisher	price
1	축구의 역사	굿스포츠	7000
2	축구아는 여자	나무수	13000
3	축구의 이해	대한미디어	22000
4	골프 바이블	대한미디어	35000
5	피겨 교본	굿스포츠	8000
6	역도 단계별기술	굿스포츠	6000
7	야구의 추억	이상미디어	20000
8	야구를 부탁해	이상미디어	13000
9	올림픽 이야기	삼성당	7500
10	Olympic Champions	Pearson	13000

그림 3-24 상관 부속질의의 데이터 예

두 개 이상 테이블에서 SQL 질의

❖ 집합 연산_도서를 주문한 고객을 알고 싶다

■ 합집합 UNION, 차집합 MINUS, 교집합 INTERSECT

질의 3-32 대한민국에서 거주하는 고객의 이름과 도서를 주문한 고객의 이름을 보이시오.

```
SELECT    name
FROM      Customer
WHERE     address LIKE '대한민국%'
UNION
SELECT    name
FROM      Customer
WHERE     custid IN (SELECT custid FROM Orders);
```

name
김연아
장미란
박세리
박지성
추신수

{고객 이름} = {대한민국에 거주하는 고객 이름} ∪ {도서를 주문한 고객 이름}

두 개 이상 테이블에서 SQL 질의

- **<여기서 잠깐> MINUS, INTERSECT 연산자**

  MySQL에는 MINUS, INTERSECT 연산자가 없으므로 다음과 같이 표현한다.

  [질의 3-32]에서 MINUS 연산을 수행한 "대한민국에서 거주하는 고객의 이름에서 도서를 주문한 고객의 이름 빼고 보이시오." 질의를 NOT IN 연산자를 사용하면 다음과 같다.

```
SELECT      name
FROM        Customer
WHERE address LIKE '대한민국%' AND
            name NOT IN (SELECT name
                         FROM       Customer
                         WHERE      custid IN (SELECT custid FROM Orders));
```

  [질의 3-32]에서 INTERSECT 연산을 수행한 "대한민국에서 거주하는 고객 중 도서를 주문한 고객의 이름 보이시오." 질의를 IN 연산자를 사용하면 다음과 같다.

```
SELECT      name
FROM        Customer
WHERE address LIKE '대한민국%' AND
            name IN (SELECT      name
                         FROM       Customer
                         WHERE      custid IN (SELECT custid FROM Orders));
```

두 개 이상 테이블에서 SQL 질의

질의(etc) 대한민국에서 거주하는 고객의 이름에서 도서를 주문한 고객의 이름 빼고 보이시오. (차집합)

```
SELECT      name
FROM        Customer
WHERE address LIKE '대한민국%'
AND name NOT IN (SELECT name
                 FROM Customer
                 WHERE custid IN (SELECT custid FROM Orders));

EXCEPT (SELECT name FROM Customer WHERE custid IN (SELECT custid FROM Orders));
```

질의(etc) 대한민국에서 거주하는 고객 중 도서를 주문한 고객의 이름을 보이시오. (교집합)

```
SELECT      name
FROM        Customer
WHERE address LIKE '대한민국%'
AND name  IN (SELECT name
              FROM Customer
              WHERE custid IN (SELECT custid FROM Orders));

INTERSECT (SELECT name FROM Customer WHERE custid IN (SELECT custid FROM Orders));
```

두 개 이상 테이블에서 SQL 질의

❖ EXISTS_주문이 있는 고객을 알고 싶다

- EXISTS는 원래 단어에서 의미하는 것과 같이 조건에 맞는 튜플이 존재하면 결과에 포함시킴. 즉 부속질의문의 어떤 행이 조건에 만족하면 참임.
- NOT EXISTS는 부속질의문의 모든 행이 조건에 만족하지 않을 때만 참임.

질의 3-33 주문이 있는 고객의 이름과 주소를 보이시오.

```
SELECT    name, address
FROM      Customer cs
WHERE     EXISTS (SELECT *
                  FROM   Orders od
                  WHERE cs.custid =od.custid);
```

name	address
박지성	영국 맨체스타
김연아	대한민국 서울
장미란	대한민국 강원도
추신수	미국 클리블랜드

두 개 이상 테이블에서 SQL 질의

❖ EXISTS_주문이 있는 고객을 알고 싶다

Customer

custid	name	address	phone
1	박지성	영국 맨체스타	000-5000-0001
2	김연아	대한민국 서울	000-6000-0001
3	장미란	대한민국 강원도	000-7000-0001
4	추신수	미국 클리블랜드	000-8000-0001
5	박세리	대한민국 대전	NULL

Orders

orderid	custid	bookid	saleprice	orderdate	
1	1	1	6000	2014-07-01	true
2	1	3	21000	2014-07-03	
3	2	5	8000	2014-07-03	true
4	3	6	6000	2014-07-04	true
5	4	7	20000	2014-07-05	true
6	1	2	12000	2014-07-07	
7	4	8	13000	2014-07-07	
8	3	10	12000	2014-07-08	
9	2	10	7000	2014-07-09	
10	3	8	13000	2014-07-10	

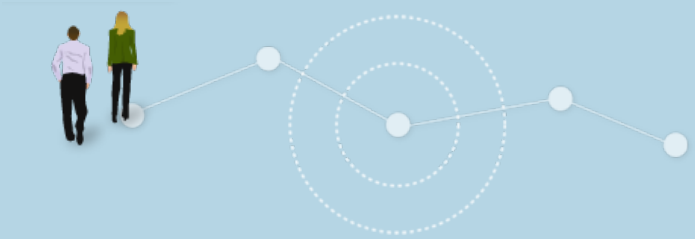
① ② ③ ④ ⑤ ✕ 없음 false

그림 3-25 **EXISTS 상관 부속질의문 데이터 예**

```
SELECT      name, address
FROM        Customer cs
WHERE       EXISTS (SELECT *
                    FROM  Orders od
                    WHERE cs.custid =od.custid);
```

데이터 정의어(DDL)

1. CREATE 문
2. ALTER 문
3. DROP 문



1. CREATE 문

- 테이블 구성, 속성과 속성에 관한 제약 정의, 기본키 및 외래키를 정의하는 명령
- PRIMARY KEY : 기본키를 정할 때 사용
- FOREIGN KEY : 외래키를 지정할 때 사용
- ON UPDATE와 ON DELETE : 외래키 속성의 수정과 투플 삭제 시 동작을 나타냄

- CREATE 문의 기본 문법

```
CREATE TABLE 테이블이름
 ( { 속성이름 데이터타입
     [NOT NULL | UNIQUE | DEFAULT 기본값 | CHECK 체크조건]
    }
     [PRIMARY KEY 속성이름(들)]
     {[FOREIGN KEY 속성이름 REFERENCES 테이블이름(속성이름)]
                [ON DELETE [CASCADE | SET NULL]
     }
 )
```

1. CREATE 문

다음과 같은 속성을 가진 NewBook 테이블을 생성하시오. 정수형은 INTEGER를 사용하며 문자형은 가변형 문자타입인 VARCHAR을 사용한다.

- bookid(도서번호)-INTEGER
- bookname(도서이름)-VARCHAR(20)
- publisher(출판사)-VARCHAR(20)
- price(가격)-INTEGER

```
CREATE TABLE    NewBook (
  bookid        INTEGER,
  bookname      VARCHAR(20),
  publisher     VARCHAR(20),
  price         INTEGER);
```

※ 기본키를 지정하고 싶다면 다음과 같이 생성한다.

```
CREATE TABLE NewBook (
  bookid      INTEGER,
  bookname    VARCHAR(20),
  publisher   VARCHAR(20),
  price       INTEGER,
  PRIMARY KEY (bookid));
```

=

```
CREATE TABLE NewBook (
  bookid      INTEGER     PRIMARY KEY,
  bookname    VARCHAR(20),
  publisher   VARCHAR(20),
  price       INTEGER);
```

1. CREATE 문

※ **bookid 속성이 없어서 두 개의 속성 bookname, publisher가 기본키가 된다면 괄호를 사용하여 복합키를 지정한다.**

```
CREATE TABLE    NewBook (
  bookname      VARCHAR(20),
  publisher     VARCHAR(20),
  price         INTEGER,
  PRIMARY KEY   (bookname, publisher));
```

bookname은 NULL 값을 가질 수 없고, publisher는 같은 값이 있으면 안 된다. price에 값이 입력되지 않을 경우 기본 값 10000을 저장한다. 또 가격은 최소 1,000원 이상으로 한다.

```
CREATE TABLE    NewBook (
  bookname      VARCHAR(20)     NOT NULL,
  publisher     VARCHAR(20)     UNIQUE,
  price         INTEGER    DEFAULT 10000 CHECK(price > 1000),
  PRIMARY KEY   (bookname, publisher));
```

1. CREATE 문

다음과 같은 속성을 가진 NewCustomer 테이블을 생성하시오.

- custid(고객번호) - INTEGER, 기본키
- name(이름) – VARCHAR(40)
- address(주소) – VARCHAR(40)
- phone(전화번호) – VARCHAR(30)

```
CREATE TABLE    NewCustomer (
 custid         INTEGER   PRIMARY KEY,
 name           VARCHAR(40),
 address        VARCHAR(40),
 phone          VARCHAR(30) );
```

1. CREATE 문

다음과 같은 속성을 가진 NewOrders 테이블을 생성하시오.

- orderid(주문번호) - INTEGER, 기본키
- custid(고객번호) - INTEGER, NOT NULL 제약조건, 외래키(NewCustomer.custid, 연쇄삭제)
- bookid(도서번호) - INTEGER, NOT NULL 제약조건
- saleprice(판매가격) - INTEGER
- orderdate(판매일자) – DATE

```
CREATE TABLE        NewOrders (
 orderid     INTEGER,
 custid      INTEGER    NOT NULL,
 bookid      INTEGER    NOT NULL,
 saleprice   INTEGER,
 orderdate   DATE,
 PRIMARY KEY (orderid),
 FOREIGN KEY (custid) REFERENCES NewCustomer(custid) ON DELETE CASCADE );
```

1. CREATE 문

- 외래키 제약조건을 명시할 때는 반드시 참조되는 테이블(부모 릴레이션)이 존재해야 하며 참조되는 테이블의 기본키여야 함
- 외래키 지정 시 ON DELETE 또는 ON UPDATE 옵션은 참조되는 테이블의 튜플이 삭제되거나 수정될 때 취할 수 있는 동작을 지정함
- NO ACTION은 어떠한 동작도 취하지 않음.

표 3-10 데이터 타입 종류

데이터 타입	설명	ANSI SQL 표준 타입
INTEGER INT	4바이트 정수형	INTEGER, INT SMALLINT
NUMERIC(m,d) DECIMAL(m,d)	전체자리수 m, 소수점이하 자리수 d를 가진 숫자형	DECIMAL(p, s) NUMERIC[(p,s)]
CHAR(n)	문자형 고정길이, 문자를 저장하고 남은 공간은 공백으로 채운다.	CHARACTER(n) CHAR(n)
VARCHAR(n)	문자형 가변길이	CHARACTER VARYING(n) CHAR VARYING(n)
DATE	날짜형, 연도, 월, 날, 시간을 저장한다.	

2. ALTER 문

- ALTER 문은 생성된 테이블의 속성과 속성에 관한 제약을 변경하며, 기본키 및 외래키를 변경함
- ADD, DROP은 속성을 추가하거나 제거할 때 사용함
- MODIFY는 속성의 기본값을 설정하거나 삭제할 때 사용함
- ADD <제약이름>, DROP <제약이름>은 제약사항을 추가하거나 삭제할 때 사용함

- ALTER 문의 기본 문법

```
ALTER TABLE 테이블이름
        [ADD 속성이름 데이터타입]
        [DROP COLUMN 속성이름]
        [MODIFY 속성이름 데이터타입]
        [MODIFY 속성이름 [NULL | NOT NULL]]
        [ADD PRIMARY KEY(속성이름)]
        [[ADD | DROP] 제약이름]
```

2. ALTER 문

NewBook 테이블에 VARCHAR(13)의 자료형을 가진 isbn 속성을 추가하시오.

ALTER TABLE NewBook ADD isbn VARCHAR(13);

NewBook 테이블의 isbn 속성의 데이터 타입을 INTEGER형으로 변경하시오.

ALTER TABLE NewBook **MODIFY** isbn INTEGER;

NewBook 테이블의 isbn 속성을 삭제하시오.

ALTER TABLE NewBook **DROP COLUMN** isbn;

NewBook 테이블의 bookid 속성에 NOT NULL 제약조건을 적용하시오.

ALTER TABLE NewBook MODIFY bookid INTEGER **NOT NULL**;

NewBook 테이블의 bookid 속성을 기본키로 변경하시오.

ALTER TABLE NewBook **ADD PRIMARY KEY(bookid);**

3. DROP 문

- **DROP 문은 테이블을 삭제하는 명령**
- **DROP 문은 테이블의 구조와 데이터를 모두 삭제하므로 사용에 주의해야 함 (데이터만 삭제하려면 DELETE 문을 사용함).**

- **DROP문의 기본 문법**

```
DROP TABLE 테이블이름
```

NewBook 테이블을 삭제하시오.

```
DROP TABLE NewBook;
```

NewCustomer 테이블을 삭제하시오. 만약 삭제가 거절된다면 원인을 파악하고 관련된 테이블을 같이 삭제하시오(NewOrders 테이블이 NewCustomer를 참조하고 있음).

```
DROP TABLE NewCustomer;
```

3. DROP 문

- **DROP 문은 테이블을 삭제하는 명령**
- **DROP문의 기본 문법**

```
DROP TABLE 테이블이름 [RESTRICT|CASCADE]
```

DROP TABLE 테이블이름 CASCADE

참조하는 테이블을 연쇄 삭제한다.

MySQL에서는 지원하지 않는다.

https://dev.mysql.com/doc/refman/8.0/en/drop-table.html

The `RESTRICT` and `CASCADE` keywords do nothing. They are permitted to make porting easier from other database systems.

`DROP TABLE` is not supported with all `innodb_force_recovery` settings. See Section 15.21.3, "Forcing InnoDB Recovery".

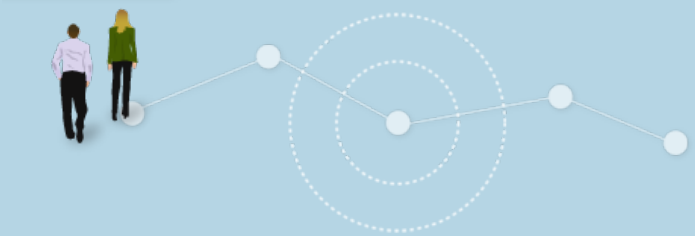
DROP TABLE 테이블이름 CASCADE CONSTRAINT -> 오라클

참조하는 테이블을 삭제하지 않고 참조 테이블의 제약 조건만 삭제한다.

원래 제약조건이 걸려서 있으면 테이블이 삭제되지 않는다.
drop table customer; -> orders 테이블에서 참조하고 있기 때문에 실행되지 않는다.
Ordres를 먼저 삭제하거나 SET_FOREIGN_KEY_CHECKS=0; 설정을 하고 지운다.

데이터 조작어(DML) – 삽입, 수정, 삭제

1. INSERT 문
2. UPDATE 문
3. DELETE 문



1. INSERT 문

- **INSERT 문은 테이블에 새로운 투플을 삽입하는 명령임.**

- **INSERT 문의 기본 문법**

```
INSERT  INTO 테이블이름[(속성리스트)]
        VALUES (값리스트);
```

Book 테이블에 새로운 도서 '스포츠 의학'을 삽입하시오. 스포츠 의학은 한솔의학서적에서 출간했으며 가격은 90,000원이다.

```
INSERT  INTO Book(bookid, bookname, publisher, price)
        VALUES (11, '스포츠 의학', '한솔의학서적', 90000);
```

bookid	bookname	publisher	price
1	축구의 역사	굿스포츠	7000
2	축구아는 여자	나무수	13000
3	축구의 이해	대한미디어	22000
4	골프 바이블	대한미디어	35000
5	피겨 교본	굿스포츠	8000
6	역도 단계별기술	굿스포츠	6000
7	야구의 추억	이상미디어	20000
8	야구를 부탁해	이상미디어	13000
9	올림픽 이야기	삼성당	7500
10	Olympic Champions	Pearson	13000
11	스포츠 의학	한솔의학...	90000

1. INSERT 문

Book 테이블에 새로운 도서 '스포츠 의학'을 삽입하시오. 스포츠 의학은 한솔의학서적에서 출간했으며 가격은 미정이다.

```
INSERT   INTO Book(bookid, bookname, publisher)
         VALUES (14, '스포츠 의학', '한솔의학서적');
```

bookid	bookname	publisher	price
1	축구의 역사	굿스포츠	7000
2	축구아는 여자	나무수	13000
3	축구의 이해	대한미디어	22000
4	골프 바이블	대한미디어	35000
5	피겨 교본	굿스포츠	8000
6	역도 단계별기술	굿스포츠	6000
7	야구의 추억	이상미디어	20000
8	야구를 부탁해	이상미디어	13000
9	올림픽 이야기	삼성당	7500
10	Olympic Champions	Pearson	13000
11	스포츠 의학	한솔의학...	90000
12	스포츠 의학	한솔의학...	90000
13	스포츠 의학	한솔의학...	90000
14	스포츠 의학	한솔의학...	NULL

1. INSERT 문

- **대량 삽입(bulk insert)이란 한꺼번에 여러 개의 투플을 삽입하는 방법임.**

수입도서 목록(Imported_book)을 Book 테이블에 모두 삽입하시오.
(Imported_book 테이블은 스크립트 Book 테이블과 같이 이미 만들어져 있음)

```
INSERT   INTO Book(bookid, bookname, price, publisher)
         SELECT bookid, bookname, price, publisher
         FROM   Imported_book;
```

bookid	bookname	publisher	price
1	축구의 역사	굿스포츠	7000
2	축구아는 여자	나무수	13000
3	축구의 이해	대한미디어	22000
4	골프 바이블	대한미디어	35000
5	피겨 교본	굿스포츠	8000
6	역도 단계별기술	굿스포츠	6000
7	야구의 추억	이상미디어	20000
8	야구를 부탁해	이상미디어	13000
9	올림픽 이야기	삼성당	7500
10	Olympic Champions	Pearson	13000
11	스포츠 의학	한솔의학...	90000
12	스포츠 의학	한솔의학...	90000
13	스포츠 의학	한솔의학...	90000
14	스포츠 의학	한솔의학...	NULL
21	Zen Golf	Pearson	12000
22	Soccer Skills	Human Kin...	15000

2. UPDATE 문

- **UPDATE 문은 특정 속성 값을 수정하는 명령이다.**

- **UPDATE 문의 기본 문법**

```
UPDATE 테이블이름
SET    속성이름1=값1[, 속성이름2=값2, ...]
[WHERE <검색조건>];
```

Customer 테이블에서 고객번호가 5인 고객의 주소를 '대한민국 부산'으로 변경하시오.

```
SET SQL_SAFE_UPDATES=0; /* Safe Updates 옵션 미 해제 시 실행 */

UPDATE    Customer
SET       address='대한민국 부산'
WHERE     custid=5;
```

custid	name	address	phone
1	박지성	영국 맨체스타	000-5000-0001
2	김연아	대한민국 서울	000-6000-0001
3	장미란	대한민국 강원도	000-7000-0001
4	추신수	미국 클리블랜드	000-8000-0001
5	박세리	대한민국 부산	NULL

2. UPDATE 문

Book 테이블에서 14번 '스포츠 의학'의 출판사를 imported_book 테이블의 21번 책의 출판사와 동일하게 변경하시오.

```
UPDATE Book
SET    publisher = (SELECT publisher
                    FROM   imported_book
                    WHERE  bookid = '21')
WHERE  bookid = '14' ;
```

bookid	bookname	publisher	price
1	축구의 역사	굿스포츠	7000
2	축구아는 여자	나무수	13000
3	축구의 이해	대한미디어	22000
4	골프 바이블	대한미디어	35000
5	피겨 교본	굿스포츠	8000
6	역도 단계별기술	굿스포츠	6000
7	야구의 추억	이상미디어	20000
8	야구를 부탁해	이상미디어	13000
9	올림픽 이야기	삼성당	7500
10	Olympic Champions	Pearson	13000
11	스포츠 의학	한솔의학...	90000
12	스포츠 의학	한솔의학...	90000
13	스포츠 의학	한솔의학...	90000
14	스포츠 의학	Pearson	NULL
21	Zen Golf	Pearson	12000
22	Soccer Skills	Human Kin...	15000

2. UPDATE 문

- **UPDATE 문은 특정 속성 값을 수정하는 명령이다.**

- **UPDATE 문의 기본 문법**

```
UPDATE 테이블이름
SET       속성이름1=값1[, 속성이름2=값2, ...]
[WHERE <검색조건>];
```

```
SET SQL_SAFE_UPDATES=0; /* Safe Updates 옵션 미 해제 시 실행 */

UPDATE     Customer
SET        address='대한민국 부산'
WHERE      custid=5;

-- 실행 안되는 구문
SET SQL_SAFE_UPDATES=1;

UPDATE     Customer
SET        address='대한민국 부산'
WHERE      address like '대한%'

Update     Customer
SET        address='대한민국 대구 '
```

custid	name	address	phone
1	박지성	영국 맨체스타	000-5000-0001
2	김연아	대한민국 서울	000-6000-0001
3	장미란	대한민국 강원도	000-7000-0001
4	추신수	미국 클리블랜드	000-8000-0001
5	박세리	대한민국 부산	NULL

3. DELETE 문

- **DELETE 문은 테이블에 있는 기존 투플을 삭제하는 명령**

- **DELETE 문의 기본 문법**

```
DELETE FROM     테이블이름
[WHERE   검색조건];
```

3. DELETE 문

Book 테이블에서 도서번호가 11인 도서를 삭제하시오.

```
DELETE FROM Book
WHERE  bookid = '11';
```

bookid	bookname	publisher	price
1	축구의 역사	굿스포츠	7000
2	축구아는 여자	나무수	13000
3	축구의 이해	대한미디어	22000
4	골프 바이블	대한미디어	35000
5	피겨 교본	굿스포츠	8000
6	역도 단계별기술	굿스포츠	6000
7	야구의 추억	이상미디어	20000
8	야구를 부탁해	이상미디어	13000
9	올림픽 이야기	삼성당	7500
10	Olympic Champions	Pearson	13000
12	스포츠 의학	한솔의학...	90000
13	스포츠 의학	한솔의학...	90000
14	스포츠 의학	Pearson	NULL
21	Zen Golf	Pearson	12000
22	Soccer Skills	Human Kin...	15000

질의 3-50　모든 고객을 삭제하시오.

```
DELETE      FROM      Customer;
```

```
27  14:36:08  DELETEFROMCustomer  Error Code: 1451. Cannot delete or update a parent row: a foreign key constraint fails (`madang`.`orders`,...  0.125 sec
```

3. DELETE 문

NewCustomer 테이블 생성

```
CREATE TABLE    NewCustomer (
custid          INTEGER   PRIMARY KEY,
name            VARCHAR(40));
INSERT INTO NewCustomer VALUES(1, '손흥민');
INSERT INTO NewCustomer VALUES(2, '김철수');
INSERT INTO NewCustomer VALUES(3, '박지성');
```

3. DELETE 문

NewCustomer의 custid=1 삭제

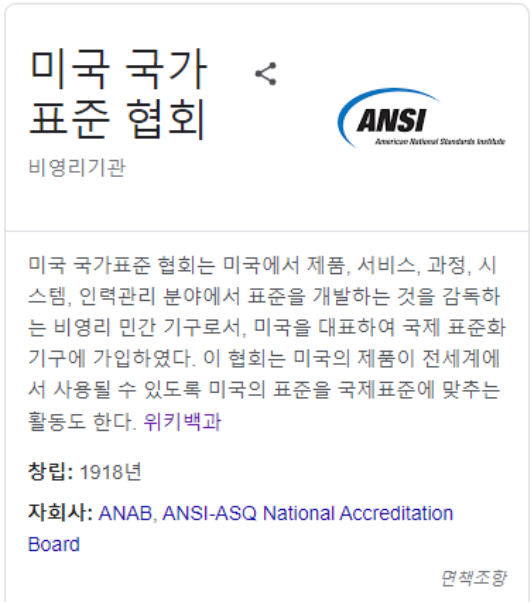
DELETE FROM NewCustomer WHERE custid=1;

-> NewOrders의 row 중에서 custid=1 인 것도 연쇄적으로 지워진다.

FOREIGN KEY (custid) REFERENCES NewCustomer(custid) ON DELETE CASCADE;

DROP TABLE NewCustomer;

-> 연쇄적으로 NewOrders는 삭제되지 않는다. 의존하는 객체(뷰, 제약조건, 테이블?)를 삭제한다. ANSI-SQL ?



SQLD 48회 시험문제로 출제

DROP TABLE 부모 CASCADE 결과로 바른 것을 찾는 문제

제약 사항만 지워진다.

자식 테이블도 지워진다.

제약 조건 체크 해제

전체 데이터 베이스 보기

SHOW databases;

Database
information_schema
madang
mysql
performance_schema
practice1
practice2
sakila
sys
world

테이블의 제약 조건 보기

USE information_schema;

SHOW tables;

Tables_in_information_schema
ROLE_TABLE_GRANTS
ROUTINES
SCHEMA_PRIVILEGES
SCHEMATA
SCHEMATA_EXTENSIONS
ST_GEOMETRY_COLUMNS
ST_SPATIAL_REFERENCE_SYSTEMS
ST_UNITS_OF_MEASURE
STATISTICS
TABLE_CONSTRAINTS
TABLE_CONSTRAINTS_EXTENSIONS
TABLE_PRIVILEGES
TABLES
TABLES_EXTENSIONS
TABLESPACES
TABLESPACES_EXTENSIONS

SELECT * FROM table_constraints where table name='neworders';

CONSTRAINT_SCHEMA	CONSTRAINT_NAME	TABLE_SCHEMA	TABLE_NAME	CONSTRAINT_TYPE	ENFORCED
practice2	PRIMARY	practice2	neworders	PRIMARY KEY	YES
practice2	neworders_ibfk_1	practice2	neworders	FOREIGN KEY	YES

제약 조건 체크 해제

NewOrders 테이블 생성

```
CREATE TABLE     NewOrders (
orderid          INTEGER   PRIMARY KEY,
custid           INTEGER NOT NULL,
bookname         VARCHAR(40),
FOREIGN KEY (custid) REFERENCES NewCustomer(custid) ON DELETE CASCADE);
INSERT INTO NewOrders VALUES(1, 1, '데이터베이스');
INSERT INTO NewCustomer VALUES(2, 2, '파이썬');
INSERT INTO NewCustomer VALUES(3, 3, '자바');
```

제약 조건 체크 해제

NewCustomer Row 또는 테이블 삭제 방법 #1

SET foreign_key_checks=0;

외래키 제약을 강제적으로 확인하지 않도록 하고 삭제

NewCustomer Row 또는 테이블 삭제 방법 #2

ALTER TABLE DROP CONSTRAINT neworders_ibfk_1;

외래키 제약을 삭제 후 시도

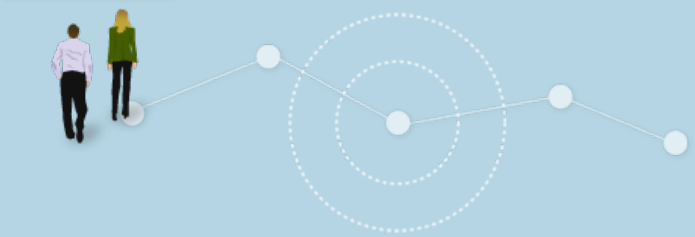
NewCustomer 테이블 삭제 방법 #3

DROP TABLE NewOrders;

NewOrders 테이블을 먼저 삭제한 후에 시도

기타 구문

1. USE 구문
2. SHOW DATABASES, TABLES 구문
3. DESC TABLE, SHOW CREATE TABLE 구문
4. LIMIT 구문



기타 구문

- **USE 구문**
  - 사용할 DB를 선택하는 구문
  - **USE practice2;**

기타 구문

- **SHOW DATABASES 구문**
  - 현재 서버에 어떤 DB가 있는지 보기
  - **SHOW DATABASES;**
- **SHOW TABLES**
  - 현재 DB에 어떤 테이블이 있는지 보기
  - **SHOW TABLES;**
- **Workbench의 Navigator로 확인 가능하지만 리눅스에 명령에서 유용하다.**

기타 구문

- **DESC TABLE 구문**
  - 테이블의 스키마를 표시
  - **DESC Book;**

- **SHOW CREATE TABLE 구문**
  - 테이블의 스키마와 제약사항 표시
  - **SHOW CREATE TABLE Book;**

기타 구문

- **LIMIT - 출력할 ROWS의 개수를 제한할 때 사용**

- **SELECT * FROM Book LIMIT 3;**
  - 결과 행 수를 3개로 제한

- **SELECT * FROM Book LIMIT 3 OFFSET 2;**
  - 전체 테이블에서 2번째 Row부터 3개 행만 표시

- **SELECT * FROM Book LIMIT 2, 1;**
  - 전체 테이블에서 2번째 Row부터 1개 행만 표시